BENCHMARK OPERATIONS

Python decides, Rust serves

A Rust data plane in front of QueryAPIGate's saved queries, with every security decision left in Python: about 8 times the calls a second per core, p99 in single milliseconds, and the audit trail that almost didn't keep up. A prototype, measured and decided go.

28 October 2026 · 6 min read

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?p=python-decides-rust-serves. This copy was made on 4 November 2026.

QueryAPIGate puts governed REST endpoints in front of SQL databases: a saved query becomes `/q/<name>`, and every call is checked, run, masked where it has to be, and recorded. Our last two benchmarks (PostgreSQL, MySQL) gave the same answer: the database isn't the limit, the gateway is. MySQL answers a per-user lookup in 0.03 ms, and the gateway adds 1.0–1.3 ms of Python around it. One worker serves 650–1,000 calls a second. Eight workers on eight cores serve about 3,000–3,500, because Python's workers don't scale well on shared cores.

The time is spread out. Logging, gunicorn and HTTP, Flask, the store reads and the driver each take 8–23% of a call. No single hot function would pay off rewriting. So we tried a different thing: a small Rust process in front of Python that serves the hot path itself, without taking any decisions away from Python.

The rule: Python decides, Rust applies

Rewriting the governance in Rust would have meant two implementations of the most security-sensitive code we have: the SQL guard, table allow-lists, column masking. Two parsers that must agree, kept in agreement by a fuzz that can only sample. We didn't want that, so the data plane doesn't decide anything.

It sits on the public port. The first time a scoped key calls a saved query, the data plane forwards the call to Python, which answers as it always does and adds a **plan** to a successful response: the statement to send, which columns to mask and how, and digests of the three rows the verdict was computed from (the key, the saved query and the connection). The data plane keeps the plan in memory and strips it from the response.

On every later call it reads those three rows again, with the same SELECTs Python uses, and serves only if they still digest the same. A key that's revoked, expired or re-scoped, or a query that's edited, changes a digest, and the call goes to Python. So does anything the prototype doesn't handle: writes, CSV, streaming, administrators, signed-in users, rate-limited keys. The data plane never answers 401 or 403 on its own, so it can never disagree with Python about one.

Python still records everything. The data plane reports each run it served in batches, and Python writes it into run history, the event log and `/metrics`, as if it had served it.

The gate: answer exactly as Python does

"Exactly" means byte for byte: the JSON body (Python's float formatting, its dates and decimals, its escaping), the headers, the error bodies, and the history row each call leaves. The parameters go into SQL as literals, the way psycopg2 and mysql-connector do it, so the quoting is the injection boundary and gets its own care.

The test that decided whether any of this counted is a **differential test**. Random calls from every kind of key (plain, masked, table-restricted, IP-restricted, rotated, expired), with valid, invalid and hostile parameters, are sent to Python directly and through the data plane, and the answers are compared whole. Partway through, grants change, a key is revoked and a query gets a new version. Any difference fails it, a refusal included. Over 22,500 calls on PostgreSQL and MySQL, it found none.

What the first benchmark found

The first full run went wrong twice before it measured anything useful, and both failures were worth having.

It filled the disk. At tens of thousands of calls a second, the request logs grow by about a gigabyte a cell, and the event log keeps every run for 24 hours: more than a million rows each cell. Each cell now starts with both empty, and the benchmark refuses to start one with less than 3 GB free.

Most of the runs went unrecorded. The data plane served 16,000–39,000 calls a second, and Python recorded about 3,000 runs a second of them. For each run reported, it parsed the entry, encoded it again, built the event and its JSON, and worked out every metric label one at a time: about 27 µs a run, under one interpreter lock. The rest were dropped from a full queue, and nothing counted them.

The fix moved work to where it's cheap. The data plane now sends each run already serialized, exactly as Python's `json.dumps` writes it, with its labels and timings. Python queues a whole batch at once, writes only the history rows it will keep, and inserts a batch's events in one statement: about 7 µs a run. Any run the data plane can't queue is counted in `/metrics`. With two Python workers behind it, every run is recorded up to about 24,000 calls a second on this laptop.

The lesson for the benchmark: a cell counts only if it recorded what it served. Every data-plane cell now reports the share of its calls that left a run in the event log. Flat out, the data plane outran that limit, so its headline figures below come from cells that kept every run.

The numbers

Measured on one laptop, each part on its own CPUs: the gateway on 8 hardware threads, the database on 4, the load generator on 8. Each figure is the median of 5 rounds, with the builds taking turns. The multiples compare the data plane with the Python build behind it, metadata cache with cache, and count Python's cores in the data plane's figure. The full report is Performance, v5.

Each side at its best on the same 8 cores (MySQL, a customer lookup, 64 callers, every run recorded):

	PYTHON, 8 WORKERS + CACHE	DATA PLANE + CACHE, 2 PYTHON WORKERS BEHIND IT
Calls a second	2,976	23,619
Gateway cores used	7.34	7.34
p99	93 ms	3.8 ms

That's **7.9 times the calls a second per core.**

PostgreSQL, at a fixed 20,000 calls a second, every run recorded: the data plane + cache used 2.97 of its 8 cores, with p99 at 1.8 ms. That's 6,734 calls a second per core, against 925 for Python + cache: **7.3 times.**

One caller at a time, a PostgreSQL lookup: 1.01 ms at p50 through Python + cache, and 0.11 ms through the data plane.

Without the metadata cache, the data plane gets 3.2–4.4× per core on lookups. Each call then reads its three store rows from the database, and that read costs the data plane what it costs Python. The cache, which both processes honour the same way, is where much of the speed comes from.

Where it doesn't help

- **Writes** are forwarded to Python, so they pay a hop: 0.35 ms more at p50, and 0.8× the calls a second per core. Serving them was never the data plane's job.
- **A 100-row page on PostgreSQL** without the cache gets 2.8× per core, because PostgreSQL itself is the limit there.
- **The first call** for each query and key goes through Python, as does every call using something the prototype forwards. These are steady-state figures: the same queries, called again and again.

What the differential test couldn't see

Before the pull request, a code review of the whole branch found real bugs the differential test missed, because it varies parameters and keys, not SQL text, sockets or failures. Two examples:

- **MySQL's time-limit hint went in at the wrong place.** Python gave its position in characters, and Rust read it in bytes, so non-ASCII text in a comment before `SELECT` could move the hint into the comment.
- **IP allow-lists could disagree on a dual-stack socket.** Rust judged `::ffff:203.0.113.9`, while Python judged `203.0.113.9`.

Both are fixed and tested, along with seven more. The differential test is necessary, not sufficient.

What happens next

It's a prototype. It's built from the repository, isn't in the Docker image or the Helm chart, and is marked experimental. The decision it was built to reach was go: it cleared the 3× bar we set in advance on every lookup, with p99 lower everywhere.

Two things come before it ships:

- **Shipping it**: the image, the chart, and which of what it forwards to serve next.
- **A store for runs at its rate.** Every run recorded at 20,000 calls a second is 1.7 billion rows a day, more than SQLite or the PostgreSQL store should hold. That's the next design question.

To try it now: Try the Rust data plane. The design and the decision are in ADR 0028.